

Vega-Lite

Composing and Interacting with Views

Information Visualization · Lecture 6

Combine views with layer, facet, and concat, then add selection and filtering.

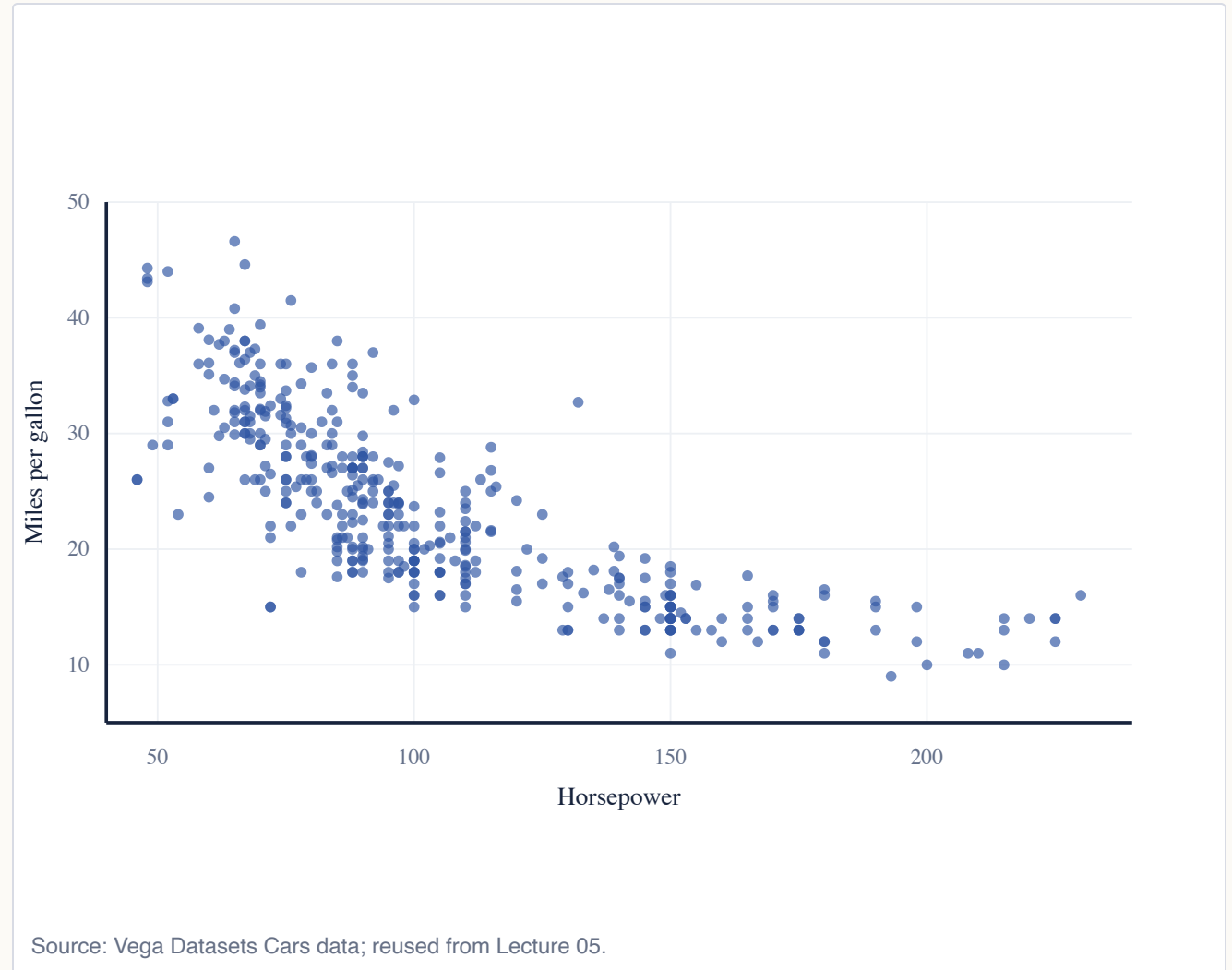
One scatterplot answers one association question

It already supports

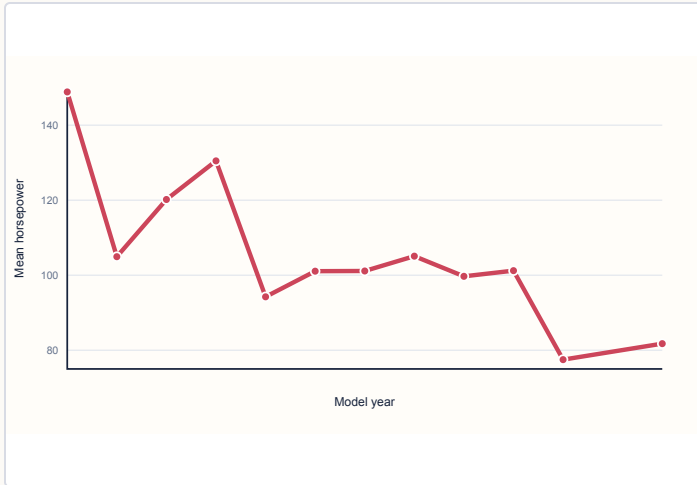
How are Horsepower and MPG associated across cars?

When the task expands

- How does the association differ across Origins?
- How can we inspect the association and the Horsepower distribution together?



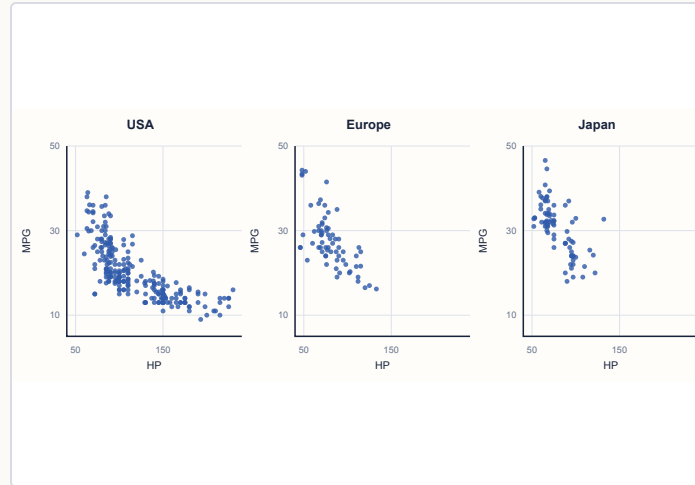
Some tasks benefit from combining views



Read a trend and its yearly values

Align points with the yearly mean Horsepower line.

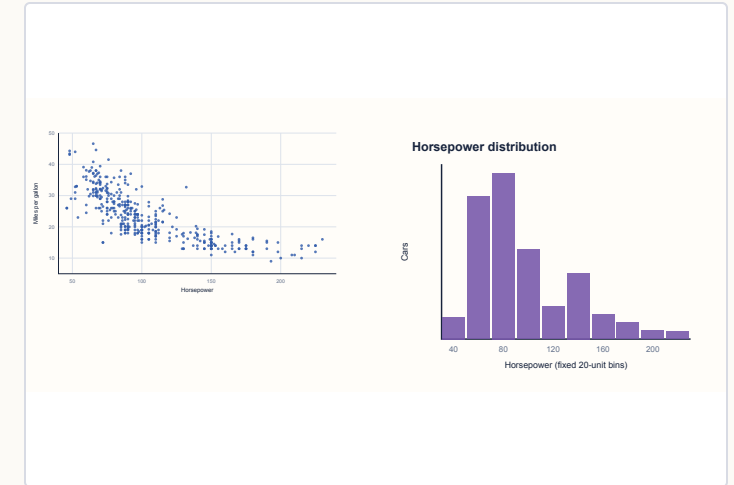
Same frame: aligned positions.



Compare the association across Origins

Repeat the Horsepower–MPG scatterplot for each Origin.

Repeated frames: comparable groups.



Inspect association and distribution

Place a Horsepower histogram beside the scatterplot.

Separate frames: complementary views.

Source: [Vega Datasets · Cars](#). Extensions of Lecture 5's line chart, scatterplot, and histogram.

Today's agenda

- | | | |
|----|-------------------------------|--|
| 01 | Composition problem | How should several views share space and data? |
| 02 | Layer · facet · concat | Match operators to analytical relationships. |
| 03 | Interaction | Declare parameters, selections, and responses. |
| 04 | Coordinated views | Let one view filter and update another. |

The composition problem connects views to a task

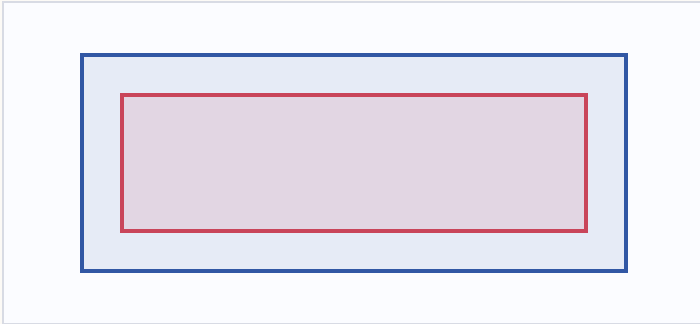
The composition problem is deciding how multiple visual descriptions of the same or related data should share space and data to support an analytical task.

Lecture 5 described individual charts with `data`, `mark`, and `encoding`. Combining those charts introduces three connected decisions.

Comparison	What should the viewer compare across the visual descriptions?
Data	Which records, groups, or summaries does each description need?
Space	Should the descriptions overlap, repeat across panels, or sit beside one another?

Reference: [Vega-Lite · View composition](#)

Vega-Lite names three composition methods

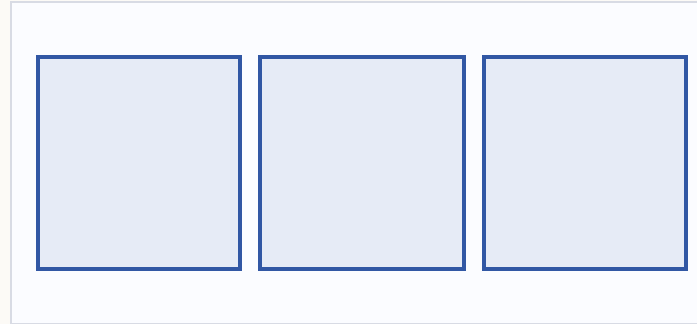


Layer

Superimposes multiple view specifications in the **same plotted frame**.

Line and points share the Year and mean Horsepower axes.

Property: `layer`

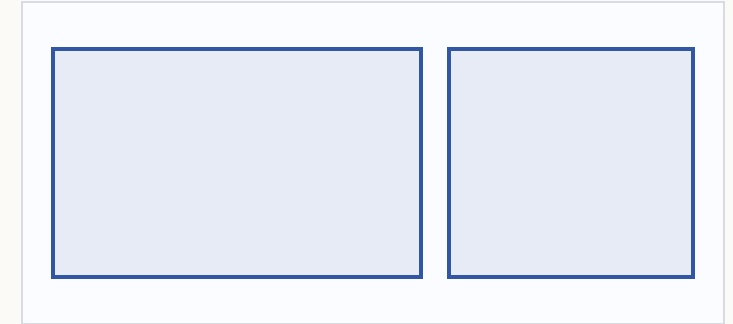


Facet

Partitions data by one or more fields and applies **one shared view specification** to each subset.

Each Origin receives the same scatterplot structure.

Property: `facet`



Concatenation

Arranges view specifications horizontally or vertically in **separate coordinate spaces**.

Views may use different marks, fields, or summaries.

`hconcat` / `vconcat`

Horizontal / vertical arrangement

Definitions: Vega-Lite [Layer](#), [Facet](#), and [Concatenation](#). Diagrams: course schematics.

The comparison guides the composition method

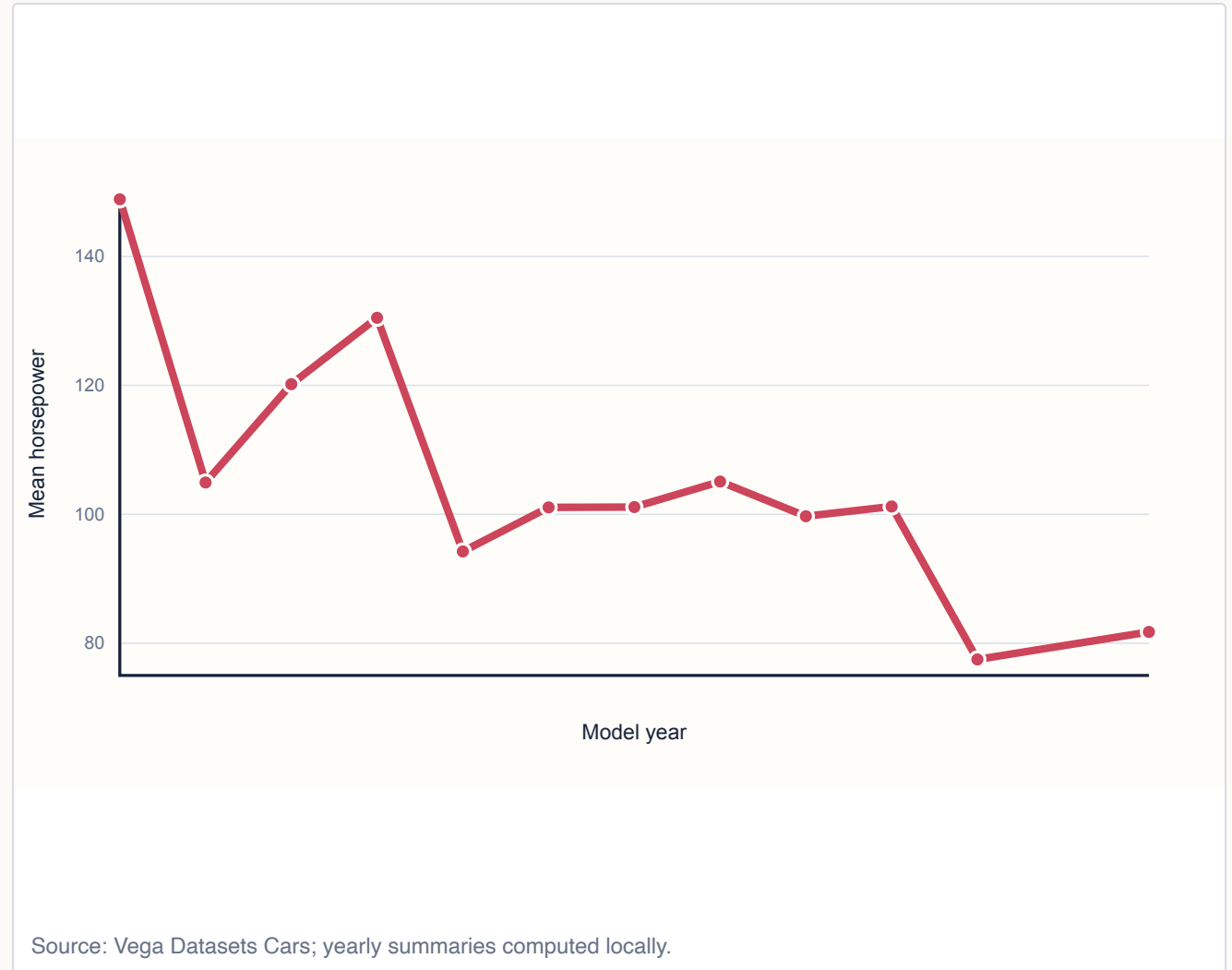
Comparison and question	Method	Why it fits
Values versus a reference How far is each car's MPG above or below a target?	Layer	Place points and a target line on the same MPG scale so the gap is directly visible.
Patterns across groups How does the Horsepower–MPG association differ by Origin?	Facet	Repeat the same scatterplot for each Origin, using common scales to compare patterns.
Different aspects of the same data How is Horsepower distributed, and how does MPG vary with it?	Concat	Place the histogram beside the scatterplot. Each view keeps the axes and summary its question needs.

Layer supports direct alignment in one coordinate system

Mean Horsepower by model year

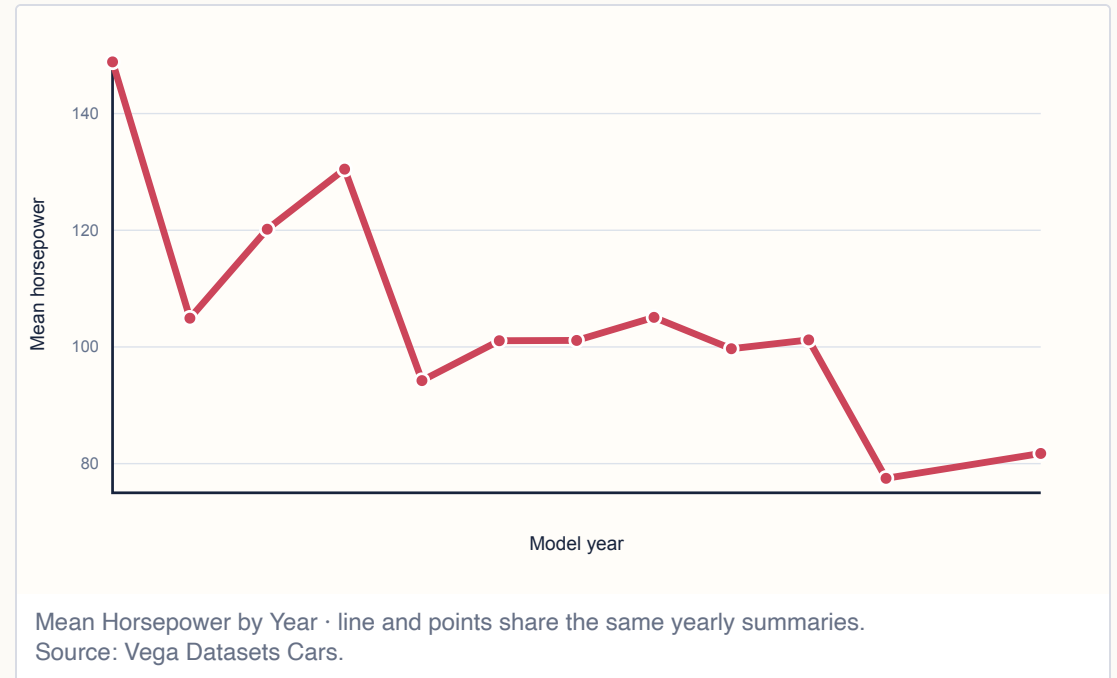
- **Line:** continuity across years
- **Points:** individual yearly summaries
- **Benefit:** exact alignment in x and y

Each value is an unweighted mean among sampled car-model records for that model year.



A layer is an array of child views

```
{
  "data": {"url": "data/cars.json"},
  "encoding": {
    "x": {"field": "Year", "type": "temporal"},
    "y": {
      "field": "Horsepower",
      "type": "quantitative",
      "aggregate": "mean",
      "scale": {"zero": false}
    },
    "color": {"value": "#cc455a"}
  },
  "layer": [
    {"mark": "line"},
    {"mark": "point"}
  ]
}
```



Shared data and encoding are inherited; each child contributes a graphical description.

A shared axis must support a meaningful comparison

Defensible overlay

An overlay is defensible when shared positions have **compatible meanings** and support the intended comparison.

Example: a line and points show the same mean Horsepower for each model year.

The axes keep the same units. The line shows the trend, and points identify yearly values.

Questionable overlay

An overlay is questionable when shared positions suggest a comparison that **incompatible measures or scales** cannot support.

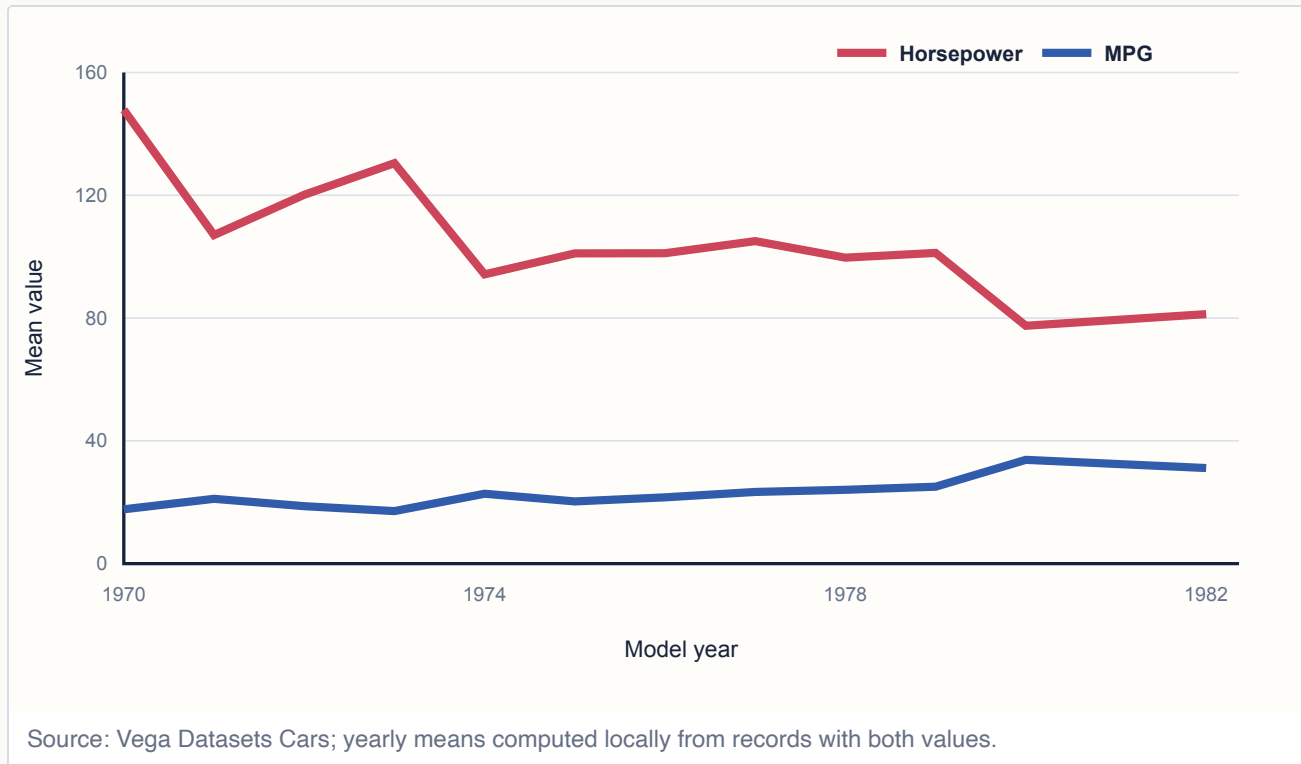
Example: mean Horsepower and mean MPG by year share one numerical y-axis.

The units differ, so the vertical gap has no common meaning. MPG also appears compressed.

Vega-Lite can combine scale domains. The designer must justify the comparison.

Reference: [Vega-Lite · Scale resolution](#). Cars examples continue on the adjacent pages.

A valid overlay can still invite an invalid comparison



Diagnose the shared axis

- **Different units:** horsepower and miles per gallon
- **Compressed pattern:** MPG is pushed near the baseline
- **Meaningless distance:** vertical separation does not compare like quantities

The lines align geometrically—but what does their vertical distance mean?

Faceting partitions data and repeats the view

Faceting partitions data by one or more fields and applies a **shared view specification** to each subset.

- Each panel shows a different data subset using the same chart structure.

Example: compare the association across Origins

Cars records

one car per record



Partition by Origin

USA · Europe · Japan

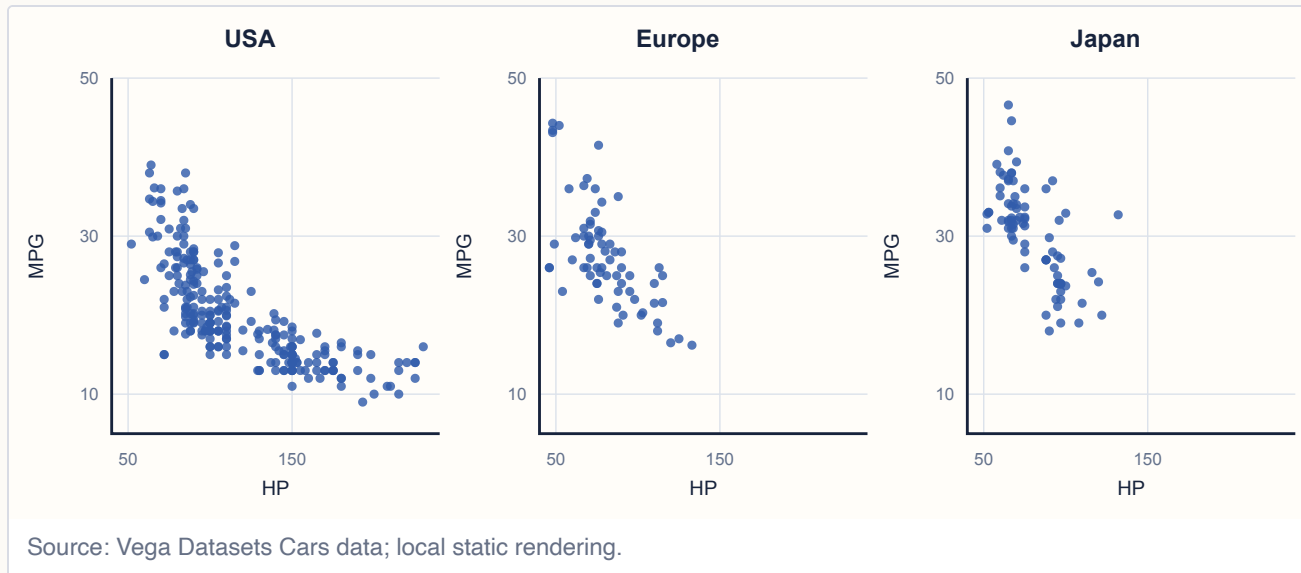


Repeat one scatterplot

Horsepower × MPG in each panel

Origin determines the panel. Horsepower and MPG keep the same encoding roles.

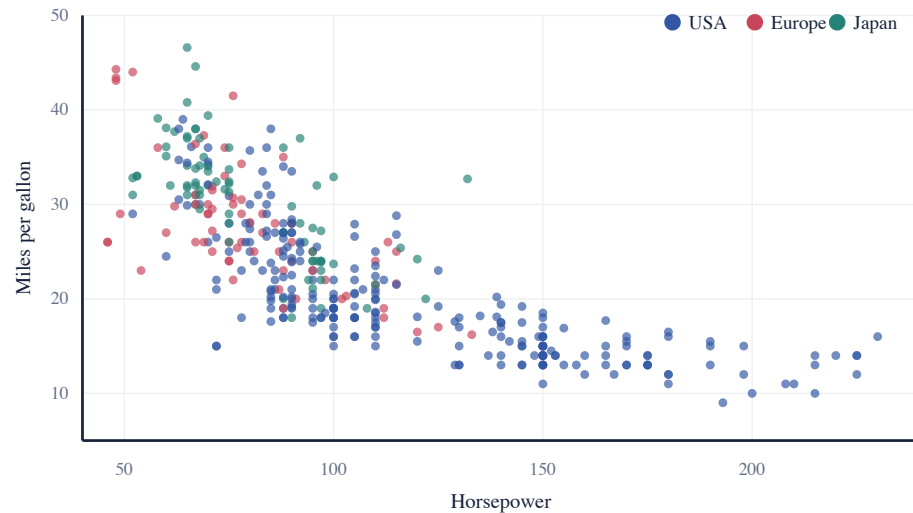
Origin becomes a spatial repetition decision



```
{
  "data": {"url": "data/cars.json"},
  "facet": {
    "column": {
      "field": "Origin",
      "type": "nominal",
      "sort": ["USA", "Europe", "Japan"]
    }
  },
  "spec": {
    "mark": "point",
    "encoding": {
      "x": {"field": "Horsepower",
        "type": "quantitative"},
      "y": {"field": "Miles_per_Gallon",
        "type": "quantitative"}
    }
  }
}
```

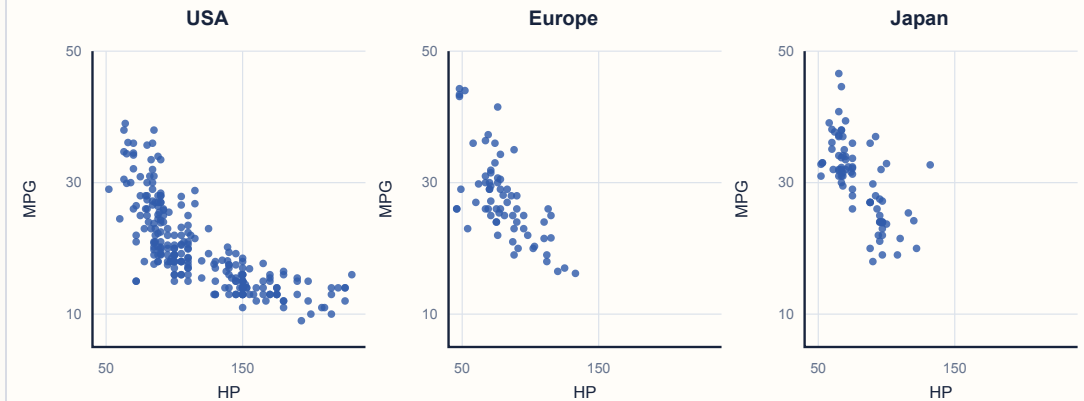
facet defines the partition; **spec** defines the view repeated for every subset.

Color and facet give Origin different design jobs



Origin → color

Preserves the overall association and group overlap—but adds color competition and occlusion.



Origin → facet

Separates groups and reduces overlap—but comparison moves across repeated panels.

Grammar makes alternatives expressible; the task determines which alternative is useful.

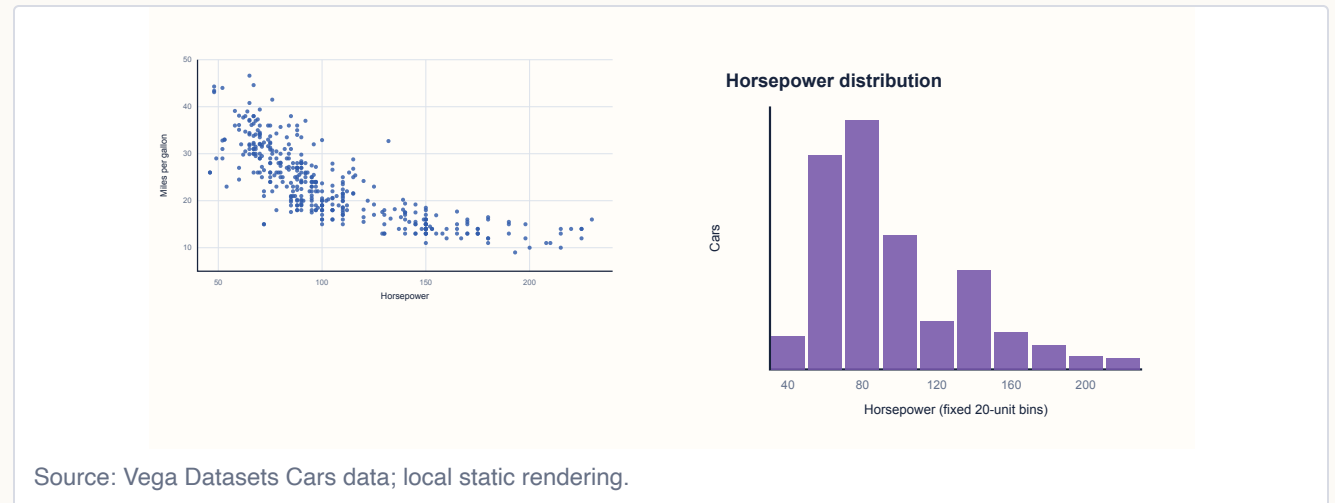
Concat keeps distinct analytical views in separate frames

One dataset, two analytical views

Some questions need both an **association** and a **distribution**, which different charts reveal.

Concat places complementary views together so readers can relate what they see in one chart to the other.

Adding interaction can strengthen that connection: an MPG selection in the scatterplot can update the Horsepower histogram.



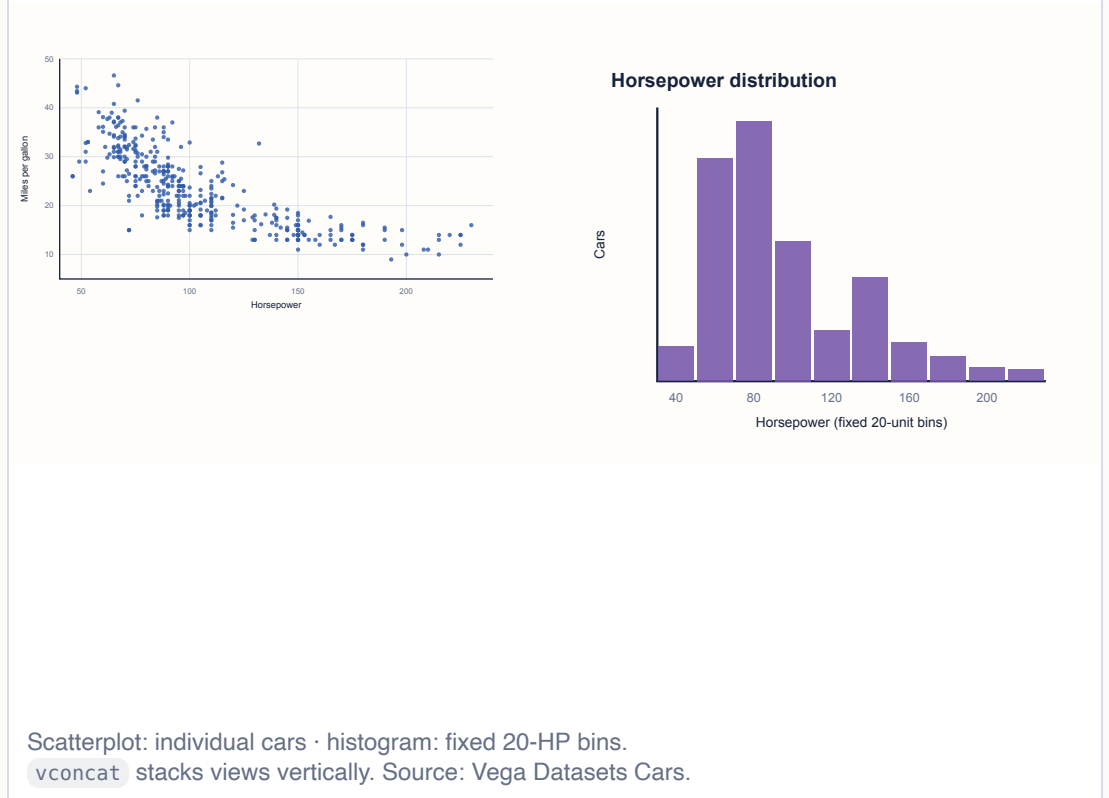
Scatterplot: point marks show the Horsepower–MPG association for individual car records.

Histogram: bar marks summarize Horsepower intervals using `bin` + `count`.

Different marks, transforms, and coordinate systems support complementary questions about the same dataset.

hconcat places child views in separate frames

```
{
  "data": {"url": "data/cars.json"},
  "transform": [{"filter": {
    "field": "Miles_per_Gallon", "valid": true}}],
  "hconcat": [
    {"mark": "point",
     "encoding": {
       "x": {"field": "Horsepower",
              "type": "quantitative"},
       "y": {"field": "Miles_per_Gallon",
              "type": "quantitative"}
     }
    },
    {"mark": "bar",
     "encoding": {
       "x": {"field": "Horsepower",
              "bin": {"step": 20,
                      "extent": [40, 240]}},
       "y": {"aggregate": "count"},
       "color": {"value": "#7656aa"}
     }
    }
  ]
}
```



Practice: choose the spatial relationship

Observed values + reference line

Same, repeated, or separate
frame?

Same association across Origins

Same, repeated, or separate
frame?

Association beside a distribution

Same, repeated, or separate
frame?

same frame → **layer** · repeated frame → **facet** · separate frames → **concat**

Interaction begins with analytical intent

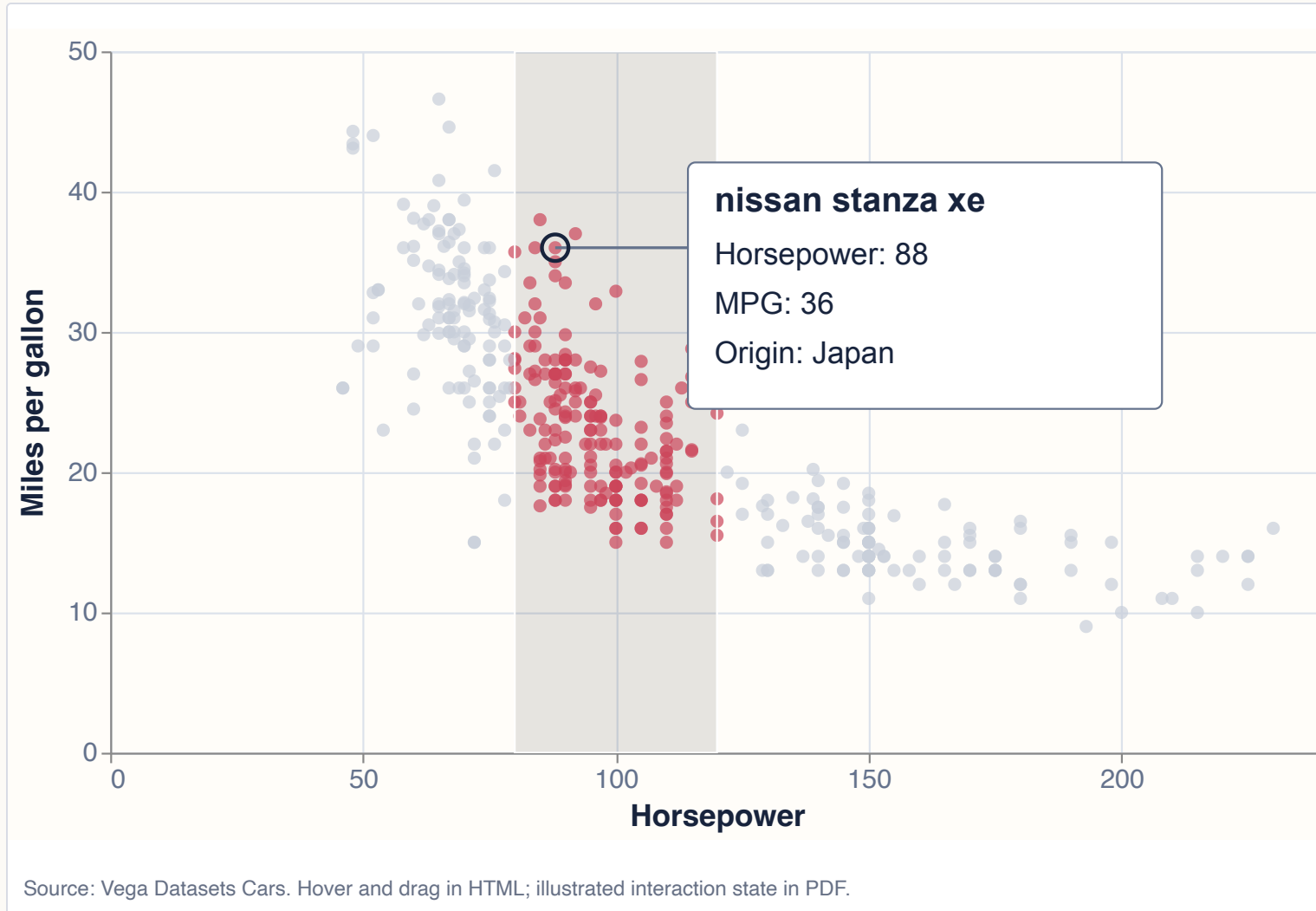
Interaction lets viewers adjust the view as their analytical questions change.

An **analytical intention** is the viewer's goal; a **user action** is an input used to pursue that goal.

Analytical intention	What the viewer wants to do	Possible user actions
Inspect	Learn details about one record.	Hover over a mark, or click for details.
Select	Define records or a value range.	Click marks, or drag across a range.
Compare	Judge a subset against its context.	Click or drag to highlight a subset.
Filter	Focus on a chosen subset.	Drag a range, or choose a menu value.

One intention can use different actions; the interface defines what each action does.

One view supports different analytical intentions



Inspect

Which car is this?

Hover over a point for its name and attributes.

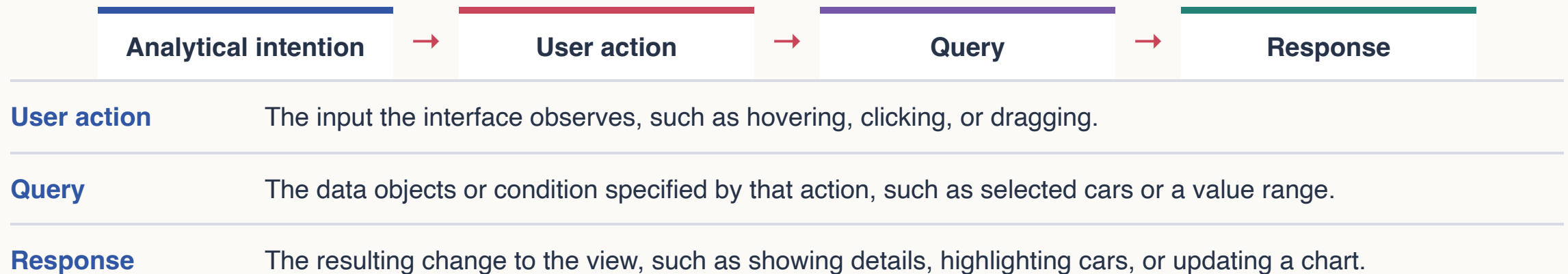
Compare

How does MPG for 80–120 Horsepower cars compare with all cars?

Drag horizontally across 80–120. Matching cars stand out; others remain visible.

From analytical intention to a visual response

An analytical intention guides the choice of action. The interface maps that action to a query and response.



Compare mid-power cars with all cars → drag across Horsepower 80–120 → query cars in that range → highlight matches while keeping all cars visible.

In Vega-Lite, the specification declares the query and response. The runtime handles the input and updates the view.

Common interactions

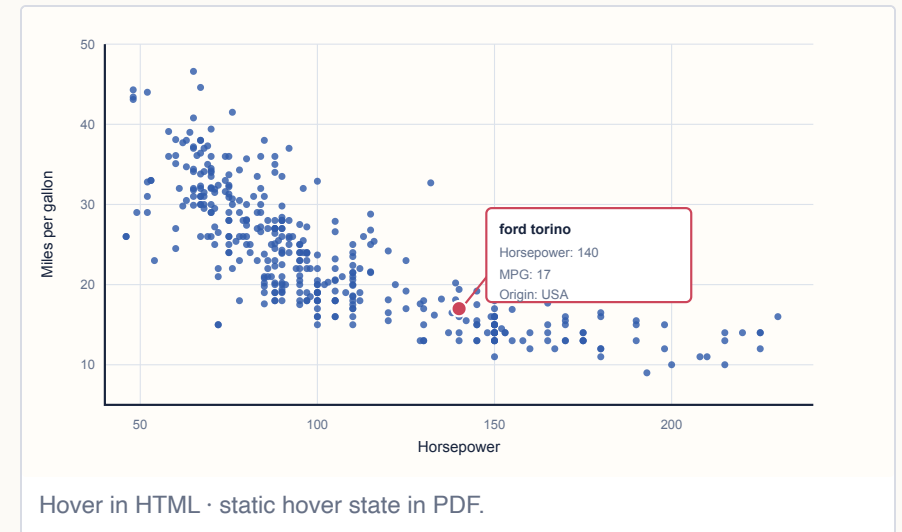
Each pattern connects an analytical intention to a user action, query, and response.

Interaction pattern	Analytical intention	User action	Query	Response
Tooltip inspection	Inspect	Hover	Car under the pointer	Show its name and attributes.
Click highlighting	Select and compare	Click	Selected cars	Highlight selected cars; retain others as context.
Brushing and linked filtering	Select and filter	Drag	Selected MPG range	Filter and recompute the Horsepower histogram.

Examples: Vega-Lite [Tooltip](#) and [Parameters](#).

Tooltip reveals details without creating a selection

```
{
  "data": {"url": "data/cars.json"},
  "mark": "point",
  "encoding": {
    "x": {
      "field": "Horsepower",
      "type": "quantitative"
    },
    "y": {
      "field": "Miles_per_Gallon",
      "type": "quantitative"
    },
    "tooltip": [
      {"field": "Name"},
      {"field": "Horsepower"},
      {"field": "Miles_per_Gallon"},
      {"field": "Origin"}
    ]
  }
}
```



Tooltip is an encoding for inspection; a basic tooltip needs no selection parameter.

Use a parameter when interaction creates named state

```
{
  "data": {"url": "data/cars.json"},
  "params": [
    {
      "name": "chosen",
      "select": "point"
    }
  ],
  "mark": "point",
  "encoding": {
    "x": {"field": "Horsepower", "type": "quantitative"},
    "y": {"field": "Miles_per_Gallon", "type": "quantitative"},
    "opacity": {
      "condition": {"param": "chosen", "value": 0.95},
      "value": 0.18
    }
  }
}
```

params

Defines named state:
`chosen` .

select

Updates that state from point clicks.

condition

Reads `chosen` to control opacity.

`data` , `params` , `mark` , and `encoding` are sibling properties of one single-view specification.

Use a parameter when interaction creates state that another part of the specification must reference.

Point selection drives conditional highlighting

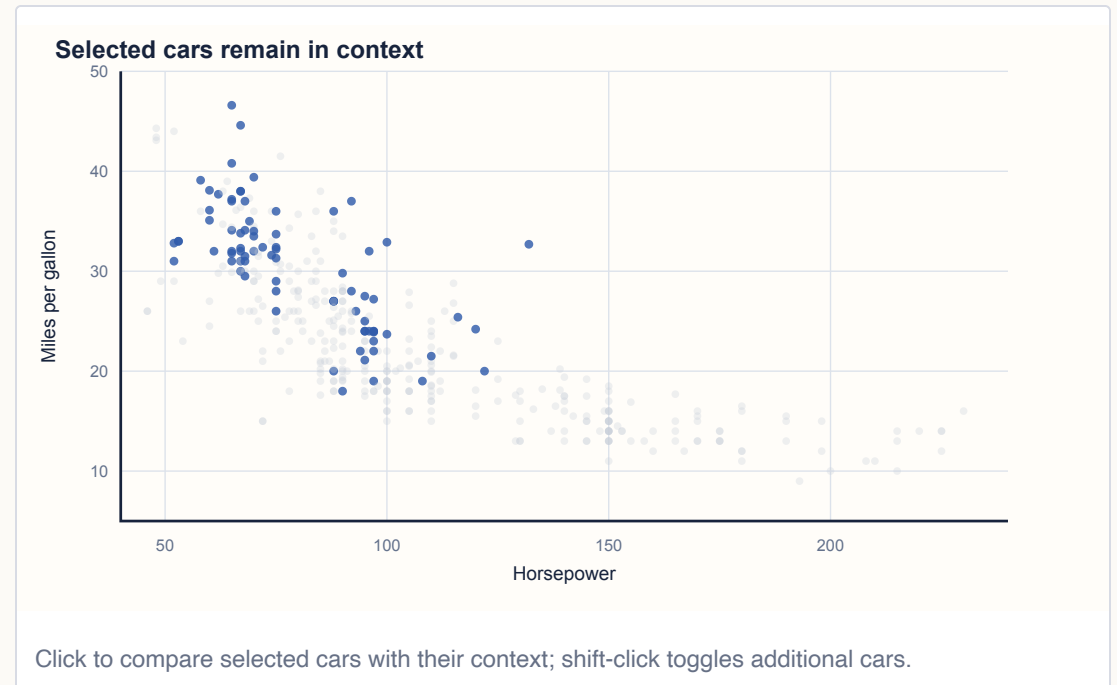
A **point selection** identifies discrete records or values.

A **condition** uses that query to control appearance.

```
"params": [  
  {"name": "chosen", "select": "point"}  
],  
"encoding": {  
  "opacity": {  
    "condition": {  
      "param": "chosen", "value": 0.95  
    },  
    "value": 0.18  
  }  
}
```

`chosen` names the query in `params`.

`condition.param` references it to control opacity.



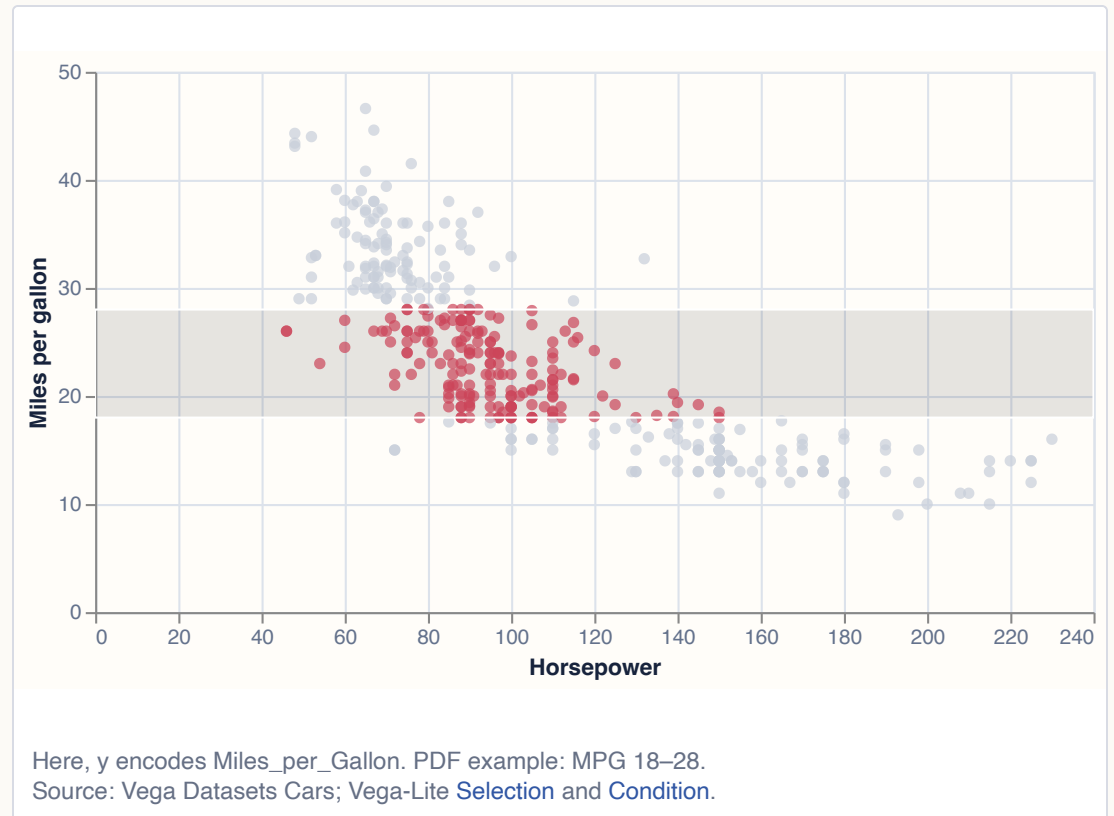
Selected cars use opacity 0.95; other cars use 0.18. All records remain available.

Source: Vega-Lite Selection parameters and Condition; Vega Datasets Cars.

Interval selection drives conditional highlighting

An **interval selection** identifies records within a continuous range of encoded values.

```
"params": [{
  "name": "brush",
  "select": {
    "type": "interval",
    "encodings": ["y"]
  }
}],
"encoding": {
  "color": {
    "condition": {
      "param": "brush", "value": "#cc455a"
    },
    "value": "#c8cfda"
  }
}
```



`encoding` maps fields to channels; `select.encodings` chooses the query channels.

`["y"]` selects an MPG range; the condition highlights matches and keeps all cars visible.

Common properties inside `select`

These properties go inside `select` ; `name` is its sibling in a `params` entry.

```
{"name": "brush", "select": {"type": "interval", "encodings": ["y"]}}
```

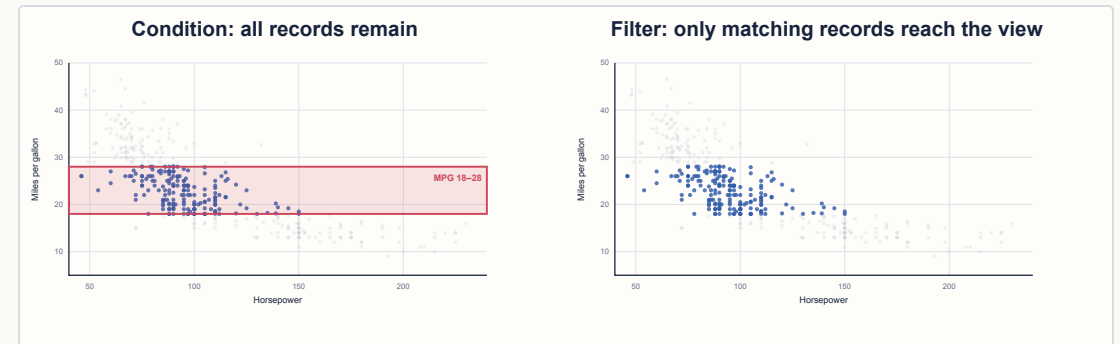
Property	Applies to	What it controls	Example
<code>type</code>	Both	Selection type.	<code>"point"</code> / <code>"interval"</code>
<code>encodings</code>	Both	Channels used for the query.	<code>["y"]</code> → MPG range (interval)
<code>fields</code>	Point	Field values used for matching.	<code>["Origin"]</code> → same Origin
<code>on</code>	Both	Event that updates the selection.	<code>"click"</code> (point example)
<code>clear</code>	Both	Event that empties the selection.	<code>"dblclick"</code> (default)
<code>toggle</code>	Point	How values are added or removed.	<code>false</code> → keep only the latest value
<code>translate</code> / <code>zoom</code>	Interval	Move / resize the existing brush.	<code>false</code> → disable that behavior

Next: use the selection query to filter records. Reference: [Vega-Lite · Selection parameters](#).

A filter changes which records reach a view

```
{
  "transform": [
    {
      "filter": {
        "param": "brush"
      }
    }
  ],
  "mark": "point",
  "encoding": {
    "x": {"field": "Horsepower", ...},
    "y": {"field": "Miles_per_Gallon", ...}
  }
}
```

`transform`, `mark`, and `encoding` are siblings inside the target child view. Shared `data` may be inherited from the parent specification.



condition → change appearance · filter → change available records

The parent composes two views; **brush** connects them

```
{
  "data": {...},
  "hconcat": [
    { // child view A
      "params": [{
        "name": "brush",
        "select": "interval"
      }],
      ...
    },
    { // child view B
      "transform": [{
        "filter": {
          "param": "brush"
        }
      }],
      ...
    }
  ]
}
```

Parent specification

shared data + horizontal composition

View A · source

params.name =
"brush"

defines the
interval query

same name



View B · target

filter.param =
"brush"

uses the query
to admit records

hconcat creates the two child views · **brush** carries the query · **filter** applies it in View B

The coordinated task links an MPG range to a distribution

For cars in a selected MPG range, how does the Horsepower distribution change?

View A · scatterplot

Analytical item: car

Input record grain: one car

Selection: MPG range

View B · histogram

Analytical item: Horsepower interval

After filter: one selected car per record

After bin + count: one interval summary

View A defines the brush; View B filters and counts

Both child views inherit the parent's Cars data inside `hconcat`.

View A · scatterplot

```
{
  "params": [{ "name": "brush",
    "select": { "type": "interval",
      "encodings": ["y"] }
  }],
  "mark": "point",
  "encoding": {
    "x": { "field": "Horsepower",
      "type": "quantitative" },
    "y": { "field": "Miles_per_Gallon",
      "type": "quantitative" }
  }
}
```

View B · histogram

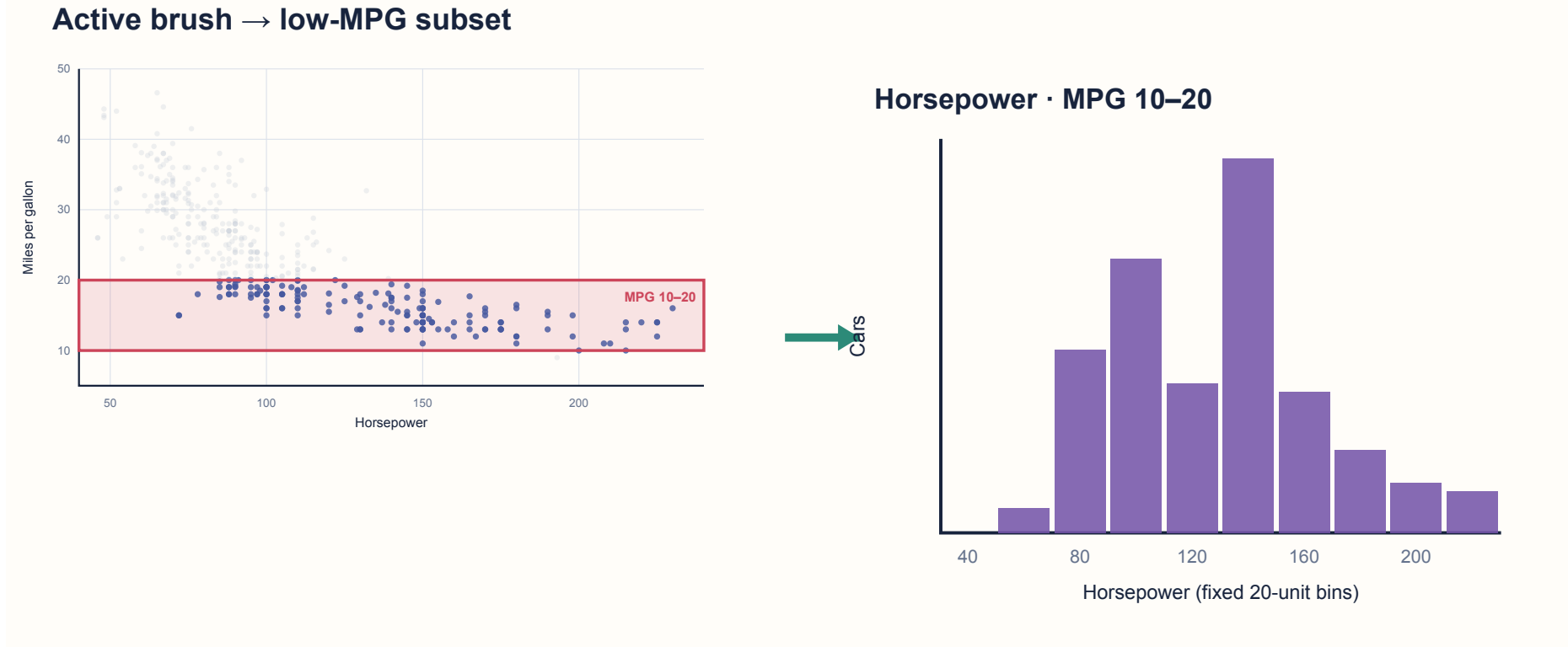
```
{
  "transform": [
    { "filter": { "param": "brush" } }
  ],
  "mark": "bar",
  "encoding": {
    "x": {
      "field": "Horsepower",
      "bin": { "step": 20, "extent": [40, 240] }
    },
    "y": { "aggregate": "count" }
  }
}
```

Filter by brush → fixed Horsepower bins → count selected cars.

Fixed 40–240 boundaries keep intervals comparable while their counts change.

Source: Vega-Lite [Selection](#), [Filter](#), and [Bin](#); Vega Datasets Cars.

Two views become one coordinated exploratory system



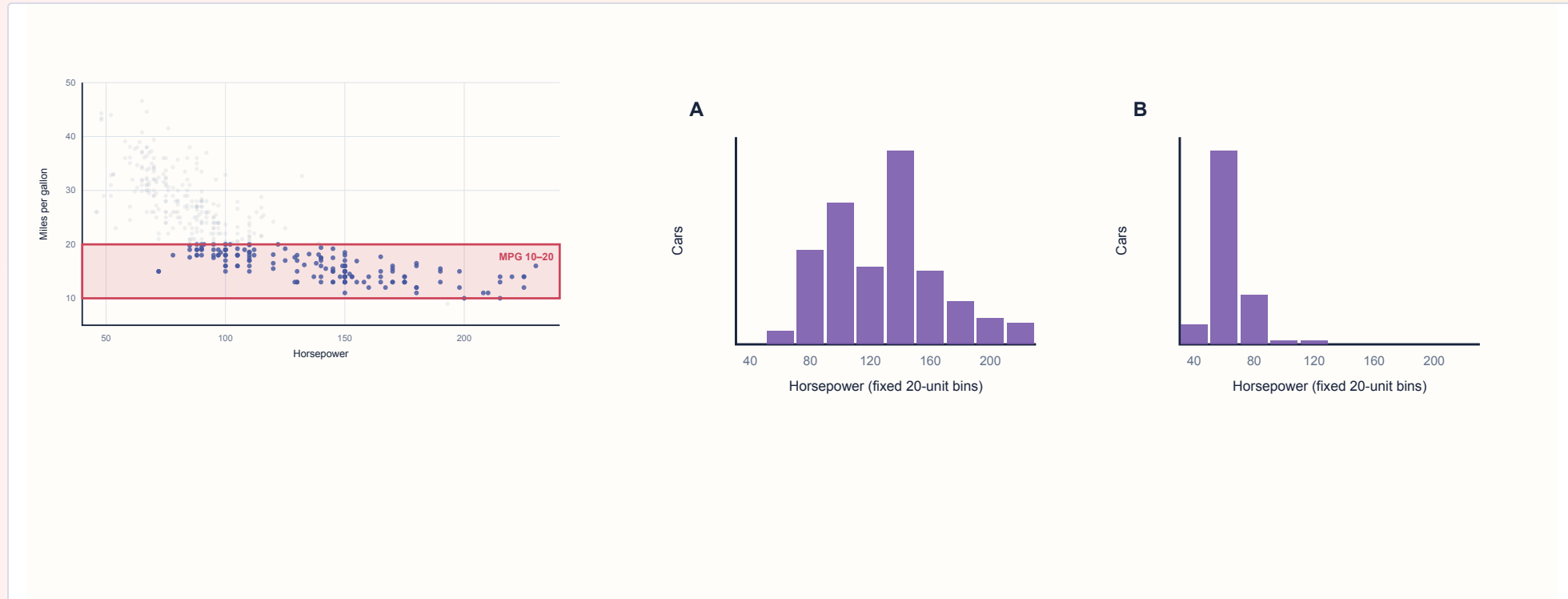
concat arranges

brush queries

filter restricts

bin + count recompute

Practice: predict the coordinated response



Which histogram is consistent with the low-MPG brush?

Answer: A · selected low-MPG car records → filter → fixed Horsepower bins → count

From a single view to coordinated views

Single view

Data → Transform → Mark + Encoding



Composition + Interaction

Arrange views and define responses.



Coordinated views

A query in one view updates another.

1 Choose the spatial relationship

Layer shares a frame; **facet** repeats a view across groups; **concat** places distinct views together.

2 Connect intention to interaction

Analytical intention → user action → query → response.

3 Choose what the response changes

condition changes appearance.

filter changes the records used for computation.